```
temp = ['a', 'b', 'c'].push('d').shift
puts temp
x = temp.upcase.next
```

```
a
```

This is a clumsy way to debug your code. If you don't want to split the call chain, you can use the tap method to slip intermediate operations into the middle of a call chain:

```
['a', 'b', 'c'].push('d').shift.tap {|x| puts x }.upcase.next
```

```
a
```

The tap method already exists on Kernel. However, it's a good exercise to imagine how you would write it yourself if it weren't already provided by Ruby:

```
class Object
  def tap
    yield self
    self
  end
end
```

Symbol#to_proc()

This exotic spell is popular among black-belt Ruby programmers. When I stumbled upon this spell for the first time, I had trouble understanding the reasoning behind it. It's easier to get there by taking one small step at a time.

Look at this code:

common_idioms/symbol_to_proc.rb

```
names = ['bob', 'bill', 'heather']
names.map {|name| name.capitalize }   # => ["Bob", "Bill", "Heather"]
```

Focus on the block—a simple "one-call block" that takes a single argument and calls a single method on that argument. One-call blocks are very common in Ruby, especially (but not exclusively) when you're dealing with arrays.

In a language such as Ruby, which prides itself on being succinct and to the point, even a one-call block looks verbose. Why do you have to go through the trouble of creating a block, with curly braces and all, just to ask Ruby to call a method? The idea of Symbol#to_proc is that you can replace a one-call block with a shorter construct. Let's start with the smallest piece of information you need, which is the name of the method that you want to call, as a symbol:

```
:capitalize
```